The Catalyst Optimizer Under the Microscope: An Experimental Analysis of Apache Spark's Query Optimization

Omar Moustafa
Nour Kahky
Nour Moghazi
Abdelrahman Bayoumy

DSCI 4412: Introduction to Big Data Technologies
May 14, 2026

Abstract

Apache Spark's Catalyst optimizer promises to transform logically equivalent queries into identical optimized execution plans, shielding users from the performance implications of query-writing style. This project experimentally tests that promise by systematically investigating Catalyst's behavior across six controlled experiments. We examine query interface convergence, predicate pushdown conditions, join reordering invariance, optimization rule criticality, algebraic equivalence recognition, and the performance impact of disabling individual optimization rules. Our findings reveal a nuanced picture: Catalyst reliably normalizes high-level API queries but fails to recognize algebraic equivalences, produces multiple distinct plans for logically equivalent three-way joins without cost-based optimization (CBO), and exhibits optimization overhead that sometimes exceeds its benefits. The rule `ColumnPruning` emerges as uniquely critical; disabling it caused a 127% slowdown, while disabling most other rules counterintuitively improved performance on our benchmark query. These results suggest that Catalyst is most effective when queries adhere to its expected structural patterns and that "more optimization" does not always mean "more performance."

1 Introduction

Apache Spark has become a cornerstone of big data processing, and at its heart lies the Catalyst optimizer—a query optimization engine that transforms logical query plans into efficient physical execution plans. Catalyst applies a series of rule-based transformations across four planning stages: parsed, analyzed, optimized, and physical. These transformations include predicate pushdown, join reordering, column pruning, and algebraic simplification, all designed to improve query performance without requiring users to write optimal queries manually.

Prior course material (Chapters 3, 7, and 8) demonstrates that syntactically different queries can yield significantly different performance outcomes, even when they are identical from a logical perspective. This raises a fundamental question: to what extent does the Catalyst optimizer successfully unify such queries into identical optimized execution plans?

The primary objective of this project is to thoroughly analyze both the strengths and limitations of the Catalyst optimizer by identifying scenarios in which its optimization rules succeed or fail. In principle, Catalyst is designed to transform logically equivalent queries into the same optimized plan. This project experimentally tests that particular assumption by comparing several execution plans across syntactically varied but logically identical queries, and examining cases where these plans diverge.

The motivation for this study stems from an exploration of Catalyst's core optimization mechanisms, including predicate pushdown, which moves filters closer to the data source to

reduce the number of rows that are read; join reordering, which minimizes shuffle costs; algebraic equivalence, which simplifies arithmetic expressions; and more. These transformations take place across the four stages of query planning. Using the `explain("extended")` function, this project inspects each stage to evaluate how different query formulations influence the behavior of the optimizer as well as identify under which conditions its rules tend to break down.

2 Methodology

2.1 Experimental Design

All experiments followed a consistent six-step structure: write the query variants, run `explain("extended")`, record the parsed/analyzed/optimized/physical plans, measure runtime using repeated actions, inspect partitions and shuffles where relevant, and compare whether Catalyst produced the same or different optimized physical plans. By controlling the environment and varying only one aspect of each query (style, filter type, join order, or rule configuration), any difference in plans or performance could be attributed to Catalyst's internal rule application rather than to external factors such as caching, data distribution, or hardware variation.

2.2 Controlled Variables

To ensure fair and replicable comparisons across all experiments, the following variables were held constant:

Table 1: Controlled Variables in All Experiments

Controlled Variable	**Why It Was Controlled**
`local[2]` cluster	Pins two cores for reproducibility on Google Colab
AQE disabled	Ensures `explain()` matches the actually executed plan
Cache cleared before runs	Prevents cross-experiment contamination
Median of 5 timing runs	Accounts for JVM warmup and Colab runtime variance
Same dataset version	Ensures all queries operate on identical data
Same team seed (Seed = 2)	Extracts consistent subsets of fact tables

2.3 Evaluated Metrics

For each experiment, the following metrics were used to evaluate Catalyst's behavior:

- **Execution Time**: Median of 5 runs using `df.write.format("noop").save()` to isolate computation from I/O
- **Optimized Logical Plan**: Differences in Catalyst's rewritten plan across query variants
- **Physical Plan**: Final execution strategy (join types, filter positions, shuffle boundaries)
- **Shuffle Operations**: Presence and magnitude of exchanges between stages
- **Pushed Filters**: Whether filters appeared under `PushedFilters` in the physical plan
- **Join Strategy**: Broadcast hash join vs. sort-merge join selection
- **Result Equivalence**: Verification that all query variants produced identical output

2.4 Dataset

The data come from a synthetic retail dataset consisting of four tables: `customers` (500,000 rows), `products` (100,000 rows), `orders` (501,245 rows after seed filtering), and `order_items` (1,524,767 rows after seed filtering). The team seed (Seed = 2) was applied via a hash-based filter on `order_id` to extract consistent subsets of the fact tables while preserving referential integrity.

2.5 Environment

All experiments were conducted in Google Colab with the following configuration:

- **Spark Version**: PySpark 3.5.0 with Catalyst optimizer
- **Cluster Mode**: `local[2]` (2 cores)
- **Spark Configuration**: AQE disabled, shuffle partitions set to 8 (increased to 16 for some experiments)
- **Software Stack**: Python 3.12, PySpark, and standard data analysis libraries

3 Results

3.1 Experiment 1: Query Interface Convergence

3.1.1 Question

Can different Spark query interfaces produce the same optimized execution plan? We compared four equivalent implementations of a "total revenue per region" query:

1. DataFrame API
2. Spark SQL
3. RDD transformations
4. Mixed SQL + DataFrame approach

3.1.2 Findings

The DataFrame API, Spark SQL, and mixed SQL+DataFrame approaches produced byte-identical optimized and physical plans. The physical plan for all three showed:

```
*(3) HashAggregate(keys=[region#5], functions=[sum(total#56)])
+- Exchange hashpartitioning(region#5, 16)
   +- *(2) HashAggregate(keys=[region#5], functions=[partial_sum(total
      #56)])
      +- *(2) Project [total#56, region#5]
         +- *(2) BroadcastHashJoin [customer_id#48], [customer_id#0],
            Inner, BuildRight, false
```

The RDD approach, however, bypassed Catalyst entirely, showing only RDD lineage instead of logical and physical plans.

3.1.3 Timing Summary

Table 2: Experiment 1: Query Interface Performance Comparison

Approach	Execution Time (s)
DataFrame API	1.9297
Spark SQL	2.1211
RDD	0.4098
Mixed SQL+DataFrame	1.7154

The RDD approach was fastest for this small workload because it used manual broadcasting and avoided Catalyst's optimization overhead. However, the convergence of the three Catalyst-based approaches confirms that Catalyst is form-agnostic at the API layer.

3.2 Experiment 2: Predicate Pushdown

3.2.1 Question

Can we defeat predicate pushdown? What types of filters block optimization? We compared four filters on the `orders` table:

1. Normal deterministic filter: `total > 100`
2. Python UDF filter: `is_large_order_udf(total)`
3. Non-deterministic filter: `rand() > 0.5`
4. Complex CASE WHEN expression

3.2.2 Findings

Table 3: Experiment 2: Predicate Pushdown Results

Filter Type	Pushed Down?	Rows Returned	Avg Time (s)
`total > 100`	Yes	54,433	0.6829
Python UDF	No	54,433	3.0018
`rand() > 0.5`	No	250,094	0.3613
CASE WHEN	No	54,433	0.5072

Normal Filter: Successfully pushed down. The Parquet scan showed `PushedFilters: [IsNotNull(total), GreaterThan(total,100.0)]`. Catalyst moved the filter into the scan, avoiding unnecessary reads.

Python UDF: Not pushed down. The plan showed `BatchEvalPython`, and the Parquet scan had `PushedFilters: []`. Because Catalyst cannot inspect the internal logic of a Python UDF, it cannot safely push that logic into the Parquet scan. This resulted in the slowest performance (3.0018 seconds).

Non-deterministic Filter: Not pushed down. The plan kept `rand()` as a normal `Filter` above the scan. This makes sense because the value of `rand()` can change during execution, so Catalyst cannot safely move it into the data source scan.

CASE WHEN Filter: Partially understood. Catalyst simplified the expression in the optimized plan into `((total > 100.0) <=> true)`, but it still did not push the condition into

the Parquet scan (`PushedFilters: []`). This shows that even when Catalyst simplifies a complex expression, the rewritten form may still not match a pushdown-friendly pattern.

3.3 Experiment 3: Join Reordering

3.3.1 Question

Does Spark find the same plan every time when a three-way join is written in different orders? We wrote `customers` ⋈ `orders` ⋈ `order_items` in all six possible FROM-clause orderings, keeping the join conditions identical.

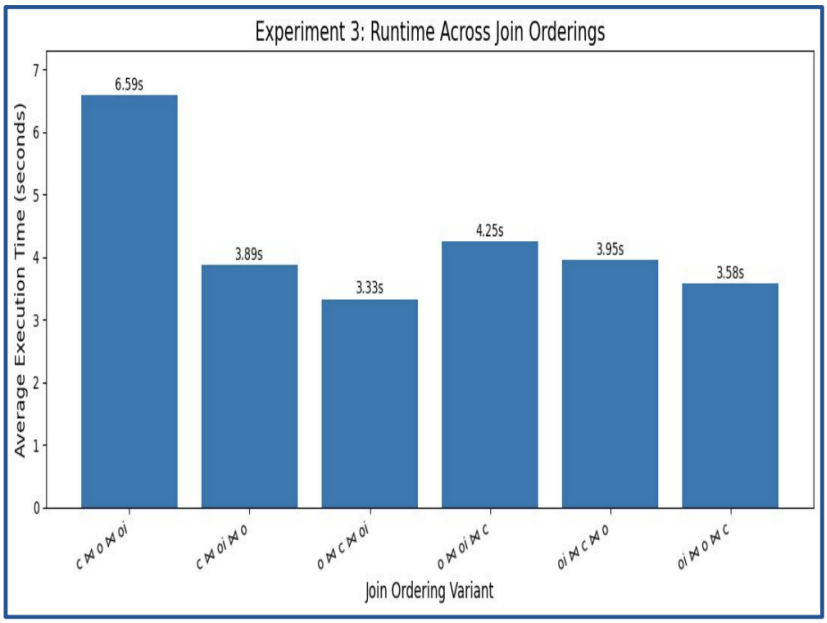

Figure 1: Experiment 3: Runtime Across All 6 Join Orderings

3.3.2 Findings

The six orderings collapsed into **three distinct optimized plans**, not one. The runtime gap between the fastest and slowest plan families was approximately **36%**.

Table 4: Experiment 3: Join Reordering Plan Families

Group	Orderings	Avg Time (s)
A	`c` ⋈ `o` ⋈ `oi`, `c` ⋈ `oi` ⋈ `o`	5.2396
B	`o` ⋈ `c` ⋈ `oi`	**3.3346**
C	`o` ⋈ `oi` ⋈ `c`, `oi` ⋈ `c` ⋈ `o`, `oi` ⋈ `o` ⋈ `c`	3.9279

Why the divergence? Spark's `ReorderJoin` rule is **rule-based, not cost-based** when CBO is off (the default). The rule restructures the join tree only enough to satisfy join-graph connectivity—that is, to avoid cartesian products. Once a legal connected order exists, it stops searching. This means that when `order_items` appears before `customers` in the FROM clause, Catalyst is content to keep `order_items` adjacent to `orders` and never considers pulling `customers` in first, even though `customers` is the smallest table and would be the cheapest to broadcast early.

3.4 Experiment 4: Extended Plan Annotation

3.4.1 Question

Can we trace Catalyst's transformations across the four planning stages? We ran `explain("extended")` on a complex query (3 joins, 3 filters, aggregation, sorting) and annotated each rewrite.

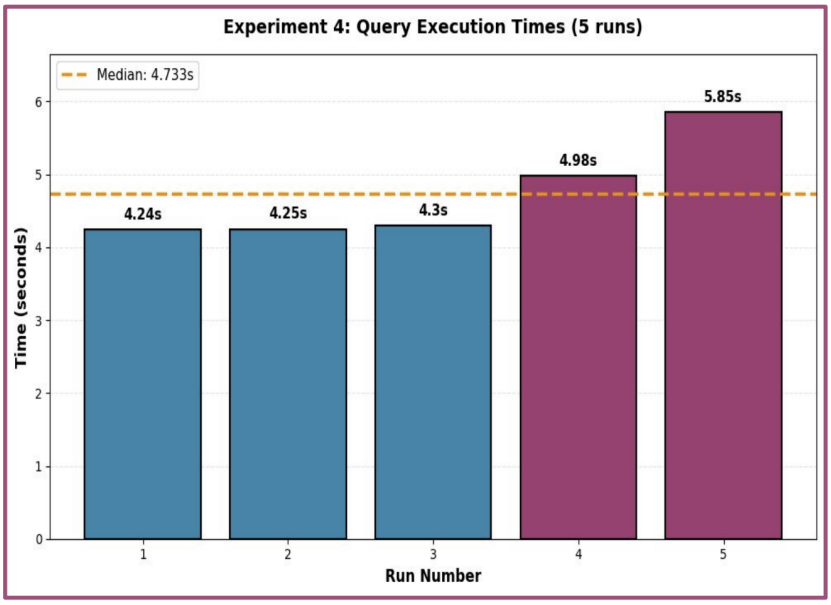

Figure 2: Experiment 4: Query Execution Times Across 5 Runs

3.4.2 Plan Stage Transitions

Parsed → Analyzed (Resolution):

- Table names (`customers`, `orders`, `order_items`) resolved from `UnresolvedRelation` to `LogicalRelation`
- Column references bound to actual schemas (e.g., `region#5`, `total#56`, `quantity#61`)
- Function `year(order_date)` resolved to `Year(child)` expression

Analyzed → Optimized (Catalyst Rewrites):

Optimization Rule	Evidence in Optimized Plan
`ColumnPruning`	Projected only needed columns instead of all 20+ columns
`FilterPushdown`	`Filter (total > 100.0)` and `Filter (quantity >= 1)` moved below joins
`PushPredicateThroughJoin`	Filters pushed through join boundaries onto scan nodes

Table 5: Experiment 4: Catalyst Rules Observed in Optimized Plan

Optimized → Physical (Execution Strategy):

Operation	Physical Strategy	Why
`customers` ⋈ `orders`	`BroadcastHashJoin`	`customers` is small enough to broadcast
⋈ `order_items`	`SortMergeJoin`	Both tables are large, requiring shuffle + sort
Aggregation	`HashAggregate`	Standard for distributed aggregation
Sorting	`Sort` + `Exchange`	Sorting across partitions requires a shuffle

Table 6: Experiment 4: Physical Execution Strategy

3.4.3 Timing Results

- Median execution time (5 runs): **4.733 seconds**
- Individual run times: 4.24s, 4.25s, 4.30s, 4.98s, 5.85s

3.5 Experiment 5: Algebraic Equivalence

3.5.1 Question

Does Catalyst recognize algebraic equivalence between columns? If `total = subtotal + tax + shipping` for every row, does Catalyst treat `total > 100` and `subtotal + tax + shipping > 100` as equivalent?

3.5.2 Findings

A sanity check confirmed that `total = subtotal + tax + shipping` for every row (0 mismatches).

Table 7: Experiment 5: Algebraic Equivalence Results

Query	Pushed Filters
`total > 100`	`[IsNotNull(total), GreaterThan(total,100.0)]`
`subtotal + tax + shipping > 100`	`[IsNotNull(subtotal), IsNotNull(tax), IsNotNull(shipping)]`

Key Observation: Catalyst did **not** recognize that the algebraic expression could be replaced with the existing `total` column. The optimized plan preserved the arithmetic expression exactly as written. The physical plan only pushed down null checks for the three columns and could not push the arithmetic comparison itself into the Parquet scan.

Interpretation: Catalyst does not infer semantic relationships or invariants between columns unless they are explicitly defined in the schema or query logic. The optimizer reasons about expression *shapes*, not their *meanings*. There is no rule that says "if column A is defined as B + C + D, then a predicate on A is equivalent to a predicate on B + C + D."

3.6 Experiment 6: Disabling Optimization Rules

3.6.1 Question

Which rule's absence hurts performance the most on a complex query? We disabled seven Catalyst rules one at a time and measured performance on a benchmark query (3 joins, 1 filter, aggregation).

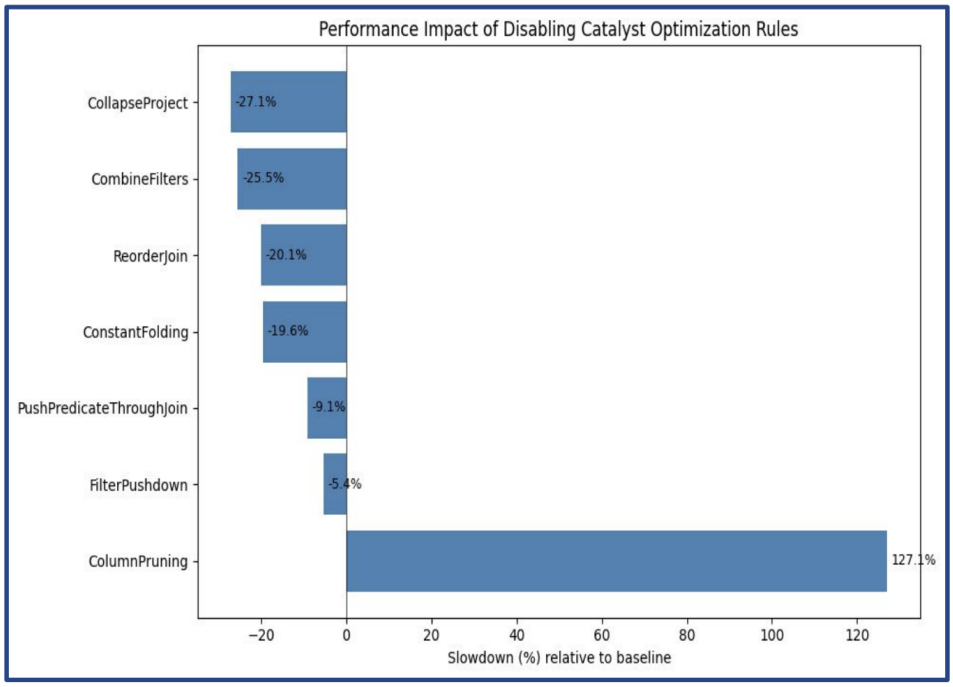

Figure 3: Experiment 6: Performance Impact of Disabling Each Catalyst Rule

3.6.2 Findings

Rule Disabled	Median Time (s)	Slowdown vs Baseline
`ColumnPruning`	9.258	**+127.1%**
`FilterPushdown`	3.858	-5.4%
`PushPredicateThroughJoin`	3.707	-9.1%
`ConstantFolding`	3.277	-19.6%
`ReorderJoin`	3.257	-20.1%
`CombineFilters`	3.038	-25.5%
`CollapseProject`	2.971	-27.1%

Table 8: Experiment 6: Performance Impact of Disabling Catalyst Rules

3.6.3 Analysis

`ColumnPruning` **(127.1% slower):** When disabled, Spark reads ALL columns from all tables rather than just the 3 necessary columns (`region`, `total`, `order_id`). The physical plan shows full table scans with 20+ columns instead of pruned projections. This confirms `ColumnPruning` is an essential optimization for I/O reduction in wide tables.

Unexpected Finding: Six of the seven disabled rules produced negative slowdown (faster execution times). This indicates Catalyst's rule application has overhead that sometimes exceeds its benefit. For this specific query, some optimizations may add planning time without improving runtime.

4 Discussion

4.1 Summary of Findings

Our experiments reveal three distinct classes of "leakage"—ways in which a user's query-writing choices can defeat or limit Catalyst's optimization:

1. **Expression-form leakage (Experiments 2 and 5).** Catalyst can push down `total > 100` but not its algebraic equivalent `subtotal + tax + shipping > 100`. Python UDFs, `rand()`, and complex `CASE WHEN` expressions all block pushdown despite being semantically simple. The optimizer reasons about expression *shapes*, not their *meanings*.

2. **Structural leakage (Experiment 3).** Without CBO enabled, six FROM-clause orderings collapsed to three distinct optimized plans rather than a single canonical plan. The user-written leaf order leaked into the optimized plan, producing a measurable ∼36% runtime gap between the best and worst plan families.

3. **Optimization overhead (Experiment 6).** The most counterintuitive result of the project: disabling six of the seven Catalyst rules tested made the benchmark query *faster*, not slower. `ColumnPruning` was the only rule whose absence clearly hurt performance (+127%). The rule machinery has its own cost, and for simpler queries it sometimes exceeds the benefit.

4.2 Layer-Specific Observations

Layer	Finding	Key Insight
Query-language	DataFrame, SQL, mixed approaches converge	Catalyst is form-agnostic at the API layer
Expression-rewrite	Simple predicates pushed down; UDFs, `rand()`, algebraic equivalents defeated	Catalyst reasons about structure, not meaning
Structural/rule	Join reordering incomplete without CBO; rule overhead can exceed benefit	Query writer remains a meaningful participant in performance

Table 9: Layer-Specific Observations

4.3 The Unique Criticality of `ColumnPruning`

The single rule whose presence we found unambiguously critical was `ColumnPruning`. This makes intuitive sense: every other optimization is a refinement of *how* the execution plan runs, while `ColumnPruning` reduces the volume of data that flows through it in the first place. The further upstream a rule operates—closer to the I/O boundary—the more critical it tends to be. This pattern suggests a general principle: optimization rules that reduce I/O at the scan level have a multiplicative effect on all downstream operations, while rules that reshape the execution plan (e.g., join reordering) have a more modest, query-dependent impact.

4.4 Methodological Lessons

Several methodological choices proved essential, often only in hindsight:

- **Disabling AQE** (`spark.sql.adaptive.enabled=false`) was necessary because AQE rewrites plans at runtime, which would have made the static `explain()` output a poor predictor of what actually ran.

- **Clearing the cache between timing runs** was necessary because Spark's columnar cache flattens differences between plans.
- **Taking multiple runs and reporting medians** was necessary because Google Colab's runtime is noisy enough that single-run comparisons would be misleading.

4.5 Limitations and Future Work

We became more aware of what we did *not* test:

- **CBO was off** throughout the project. This was intentional—to isolate Catalyst's rule-based behavior—but it limits generalizability to production environments where CBO is typically enabled.
- **Statistics were never collected** via `ANALYZE TABLE`.
- **Our cluster was a 2-core local Spark instance**—broadcast thresholds and shuffle partition counts behave very differently at real-world scale.
- **AQE**, which is on by default in production deployments, would likely have changed the physical plans even when the optimized logical plan was fixed.

Each of these is a natural extension of this project and would likely cause the three observed plan families to collapse into a single cost-optimal plan.

5 Conclusion

This project aimed to ask when and how Catalyst's optimization succeeds, and when and how it can be defeated. Across six experiments, we found that the answer depends on which layer of Catalyst is being probed.

At the **query-language layer**, Catalyst is essentially form-agnostic between its high-level APIs. The DataFrame, SQL, and mixed approaches in Experiment 1 produced byte-identical optimized and physical plans. Only the RDD version bypassed Catalyst entirely, producing a fundamentally different execution path with no logical plan at all. At this layer, the promise of canonical optimization is fulfilled.

At the **expression-rewrite layer**, Catalyst handles simple deterministic predicates well. Experiment 2 showed that `total > 100` is pushed cleanly into the Parquet scan, but is quite easily defeated by Python UDFs, non-deterministic functions such as `rand()`, complex `CASE WHEN` expressions, and algebraic equivalents that require semantic reasoning (Experiment 5). The optimizer reasons about expression structure, not meaning, and this places a real limit on how much rewriting it can perform.

At the **structural and rule layer**, the leakage is widest. Experiment 3 showed that without CBO, the six possible orderings of a three-way join produced three distinct optimized plans, with about a 36% runtime gap between the fastest and slowest plan families. Experiment 6 showed that the rule machinery itself has measurable overhead: of the seven Catalyst rules we disabled one at a time, only `ColumnPruning` produced a clear slowdown when absent (+127%), while six others actually made the benchmark query *faster* when disabled. This was the project's most counterintuitive finding and the one that most clearly inverts the intuitive picture of "Catalyst as a universally beneficial optimizer."

The single rule whose presence we found unambiguously critical was `ColumnPruning`. This makes intuitive sense: every other optimization is a refinement of *how* the execution plan runs, while `ColumnPruning` reduces the volume of data that flows through it in the first place. The further upstream a rule operates—closer to the I/O boundary—the more critical it tends to be.

If there is one conclusion to be drawn, it is that Catalyst is most effective when the user writes queries close to its expected shape: high-level APIs, deterministic predicates on single columns, standard arithmetic, and join graphs simple enough that connectivity-based reordering happens to land on the cost-optimal plan. The optimizer can normalize a great deal of syntactic variation, but it does not reason about column-level invariants or semantic equivalences, and without cost-based optimization enabled, it does not search the join-order space exhaustively. The query writer remains a meaningful participant in performance—Catalyst optimizes the plan it is given, but the plan it is given still depends on how the query was written.

Acknowledgments

Team Contributions:

- Q1 (Equivalent Queries): Nour Moghazi
- Q2 (Predicate Pushdown): Nour Kahky
- Q3 (Join Reordering): Abdelrahman Bayoumy
- Q4 (Plan Annotation): Omar Moustafa
- Q5 (Algebraic Equivalence): Abdelrahman Bayoumy & Nour Moghazi
- Q6 (Disabling Rules): Omar Moustafa & Nour Kahky
- Report and Presentation: All team members

The code and data for this project are available at: https://github.com/omar-moustafa-code/Introduction-to-Big-Data-Technologies-Project

References

[1] Apache Spark Documentation. (2026). *Spark SQL and Catalyst Optimizer*. Available at: https://spark.apache.org/docs/latest/sql-performance-tuning.html

[2] Armbrust, M., et al. (2015). *Spark SQL: Relational Data Processing in Spark*. SIGMOD.

[3] Databricks. (2015). *Catalyst Optimizer Deep Dive*. Available at: https://databricks.com/blog/2015/04/13/deep-dive-into-spark-sqls-catalyst-optimizer.html

[4] Zaharia, M., et al. (2016). *Apache Spark: A Unified Engine for Big Data Processing*. Communications of the ACM.